PetriFlow, reviewed

Architecture, seven live demos, validated loopholes, the website against the code, and the position of a static Petri-net gate relative to self-evolving-agent research.

Notes to accompany the 44-slide deck. Chapters 1 to 7 are the text chapters of the companion repository, with the key diagrams inserted. Every expected output was captured by running the code on 9-10 October 2026; every file reference was re-read by hand. Sources for chapter 6 are in research/sources.txt.

Contents: 1 first principles; 2 architecture and file map; 3 website versus code; 4 is the mathematics real; 5 loopholes, validated; 6 self-evolving agents research; 7 the 90-minute demo script.

Contents

Contents 2

Chapter 1. First Principles In Ten Minutes 4

1.1 The problem 4

1.2 The second problem: trusting the gate 4

1.3 Petri nets in one paragraph 4

1.4 The trick: a rule becomes a shape 5

1.5 The two things the project is 5

1.6 The history in one line per release (from CHANGELOG) 6

1.7 What to keep in mind for the rest of the chapters 6

Chapter 2. Architecture And The File Map 7

2.1 Two products in one repository 7

2.2 Dependency chain 7

2.3 File map, package by package 8

2.4 One tool call, end to end (diagram 13) 10

2.5 The workflow engine tick (diagram 20) 10

2.6 What the gate does NOT use at runtime 11

2.7 State between calls, per adapter 11

2.8 Reading order for the code (90 minutes) 11

Chapter 3. The Website Versus The Code 12

3.1 What the website is 12

3.2 How the "real example" is shown 12

3.3 Claims on the site that the code does NOT back 13

3.4 Claims that DO check out 13

3.5 One-sentence verdict for the meeting 13

Chapter 4. Is The Petri-Net Mathematics Real, Or Is "Petri Net" A Fancy Word? 14

4.1 What is genuinely Petri-net theory here 14

4.2 What is NOT there (and what the vocabulary suggests) 14

4.3 "Are they using the gate as a skill and Petri net as a fancy word?" 15

4.4 Where the proof vocabulary is justified 15

4.5 One demonstration that settles the question in two minutes (chapter 7, demo 3) 15

Chapter 5. Loopholes: What Is Real, Ranked, And How Each Was Validated 16

5.1 The five you can run live (bun run labs/loopholes.ts) -- all [RUN] 16

5.2 The one you can run against a server (labs/openai/03-server.ts + 03-spoof.ts) [RUN] 17

5.3 Verified by reading the code [READ] 17

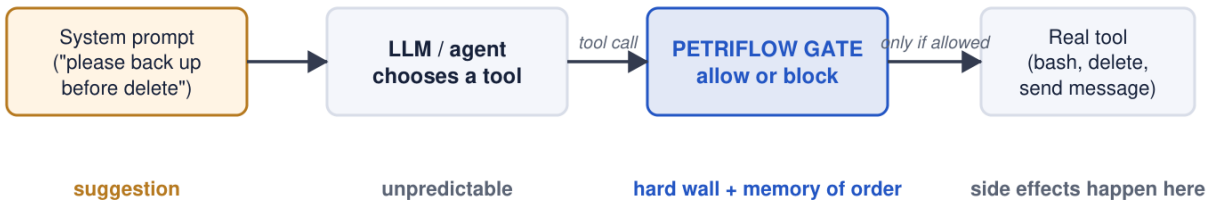
5.4 What is NOT a loophole (be fair in the meeting) 18

5.5 The one-sentence version 18
Chapter 6. Self-Evolving Agents: Where Petriflow Stands And What To Research 19
6.1 What "self-evolving agent" means (two reference surveys) 19
6.2 Landmark systems and WHAT evolves in each 19
6.3 Safety findings specific to self-evolution (why a static gate matters) 19
6.4 Formal methods and Petri nets around LLM agents (the neighbourhood PetriFlow lives in) 20
6.5 PetriFlow's public footprint 20
6.6 How far has PetriFlow gone toward self-evolving agents? Not at all, by design. 20
6.7 What can be concluded from this repo for self-evolving-agent work 20
6.8 Suggested one-paragraph position for the meeting 21
Chapter 7. The 90-Minute Review: Agenda, Demos, Commands, Expected Output 22

Chapter 1. First Principles In Ten Minutes

(The long version, with every diagram, is docs/10-full-guide-explained-from-zero.txt and pdf/PetriFlow_Explained_From_Zero.pdf. Diagrams: diagrams/00 to 10.)

1.1 The problem



The gate sits between the model and the tool. It is the only place that can say NO and remember what already happened.

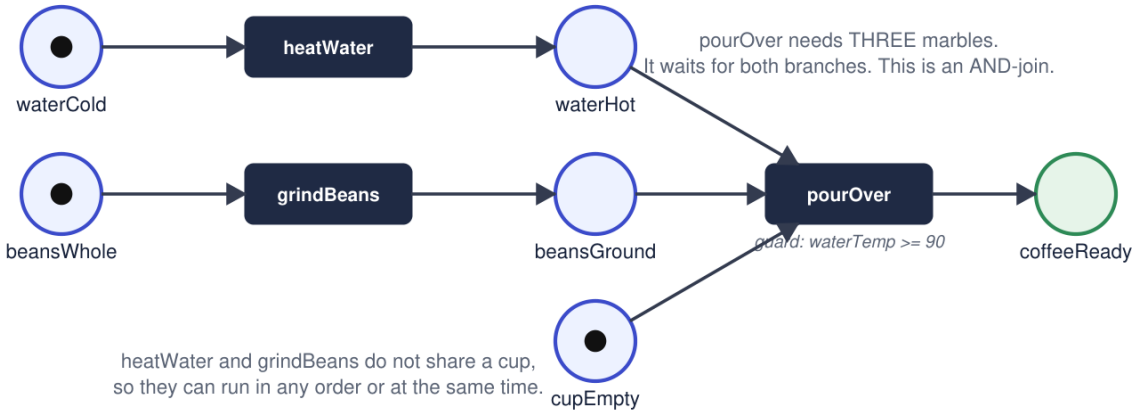
diagrams/01-where-the-rule-lives.png

An AI agent is a model in a loop: read task, choose a tool, call it, read the result, choose again. Tools have side effects (delete, deploy, send). The model's choices are not guaranteed. Four common controls all leak:

prompts	a suggestion, not a wall
deny lists	never complete (block rm, miss find -delete)
if-statements	live in the tool code, know nothing about ORDER across tools
human approval of everything	nobody reads the 40th dialog

The missing piece is a component BETWEEN the model and the tools that knows the history of calls and decides from it. (diagram 01)

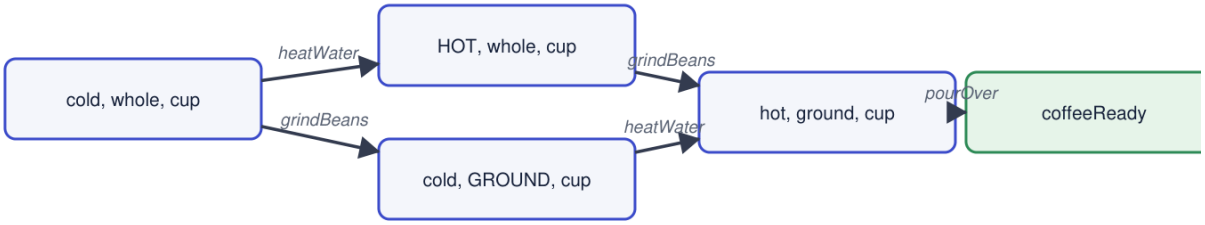
1.2 The second problem: trusting the gate



diagrams/03-coffee-net.png

If the gate is itself a pile of if-statements, you are back where you started. You want a gate whose policy you can inspect completely before anything runs. That is the reason for the Petri net: a policy small enough to enumerate every state it can be in.

1.3 Petri nets in one paragraph



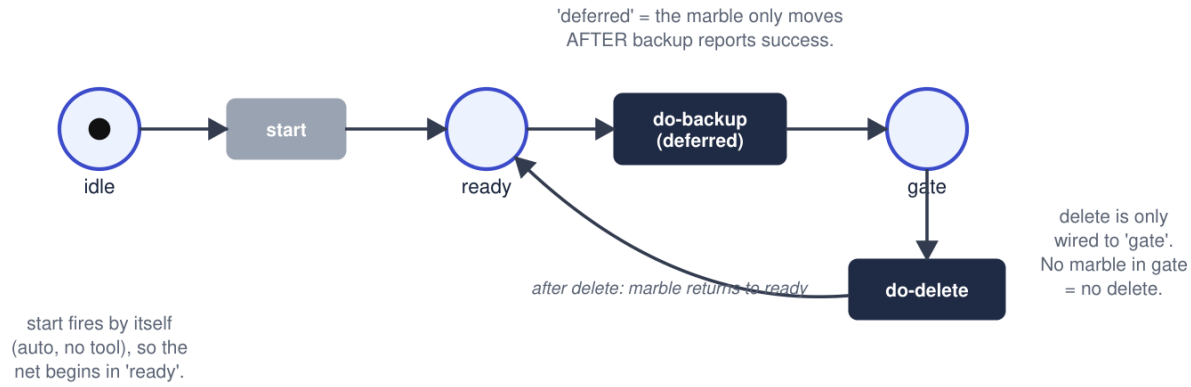
Every box is one 'marking' (a snapshot of all cups). The analyser walks every arrow from the start and collects every box it can reach. Here: 5 reachable states, 1 terminal state, and it is the good one.

diagrams/04-coffee-reachability.png

Places are cups. Tokens are marbles in cups. A transition is a machine that needs marbles in all its input cups, takes them, and puts marbles in its output cups. The marking is the count in every cup: the whole state. A transition is enabled when its inputs are filled. Reachable states are every marking you can get to by firing enabled transitions in any order. A terminal state is one where nothing can fire: fine if it is the intended end, a deadlock otherwise. An invariant is a weighted count of marbles that never changes. (diagrams 02, 03, 04, 05)

Why not a state machine or a DAG? A state machine has one marble; it cannot say "heat water AND grind beans, in either order, then pour". A DAG has no tokens, so it cannot express a budget, a lock, or "once per X". Petri nets do both with the same five words.

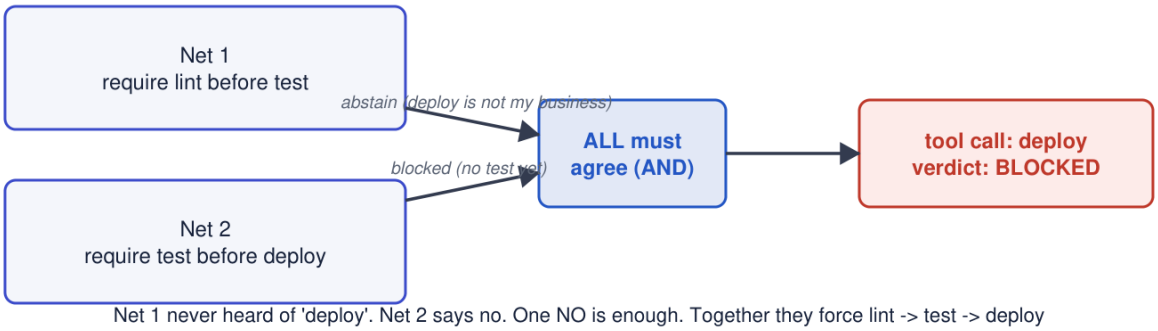
1.4 The trick: a rule becomes a shape



diagrams/06-rule-require-before.png

```
  require A before B            idle -> A fires (deferred) -> gate -> B fires -> idle. 3 states
  require human-approval before B   a MANUAL transition on a self-loop. 2 states
  block A                       A's transition reads from a cup that can never fill. 2 states
  limit A to N per session      a budget cup with N marbles. N+2 states
  limit A to N per X            budget plus a spent cup and a refill fired by X. N+2 states
  map tool.field /regex/ as name  renames a tool by its input before any net looks at it
Rules compose by AND: a call is allowed only if every net that has an opinion says yes.
Each net is tiny, so each is enumerated in microseconds. (diagrams 06 to 10, 15)
```

1.5 The two things the project is



Net 1 never heard of 'deploy'. Net 2 says no. One NO is enough. Together they force lint -> test -> deploy without either net knowing about the other. Safety rules compose by intersection.

diagrams/10-rules-compose.png

A workflow ENGINE (define a net with guards and executors, run it with a scheduler and SQLite, analyse it with a CLI) and a tool GATE (SkillNets + GateManager + adapters + a rules DSL). Both sit on the external petri-ts library, where the mathematics lives. (diagram 11)

1.6 The history in one line per release (from CHANGELOG)

0.1.0 engine first, then the gate and presets; 0.2.0 the gate becomes stateless (state in the caller's session); 0.3.0 block messages the model can act on, ruleMetadata; 0.3.1 replay from message history; 0.4.0 "failed" is not always an exception (isToolResultError). (diagram 12)

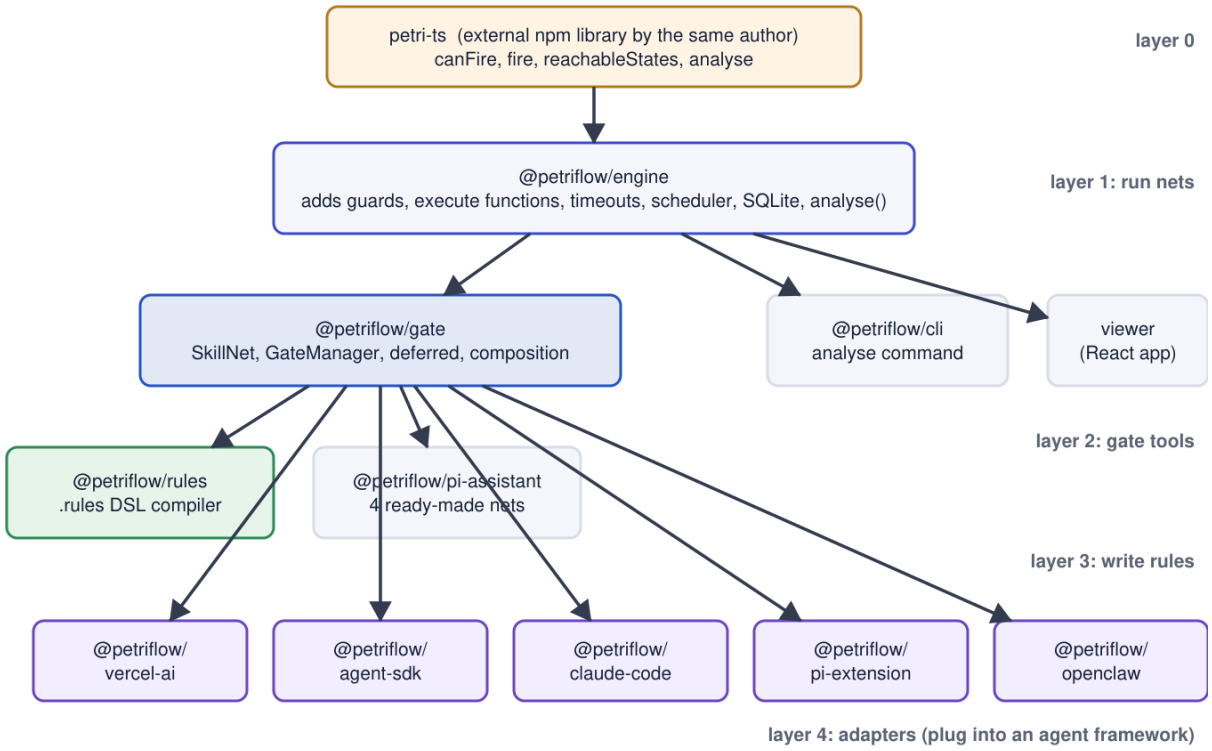
1.7 What to keep in mind for the rest of the chapters

The gate sees tool NAMES and ORDER. It does not see what tools do, what inputs mean, or whether the history it replays is true. Everything in chapters 4 and 5 follows from that.

Chapter 2. Architecture And The File Map

Everything here was read from the source of github.com/joshuaisaact/petri-flow on
10 October 2026 (main branch, commit dcd6196). Line counts are approximate.
Diagrams: diagrams/11-package-layers.png, 13-tool-call-journey.png, 16-gate-state.png,
17-rules-pipeline.png, 18-adapters.png, 20-scheduler-tick.png, 21-sqlite-persistence.png.

2.1 Two products in one repository



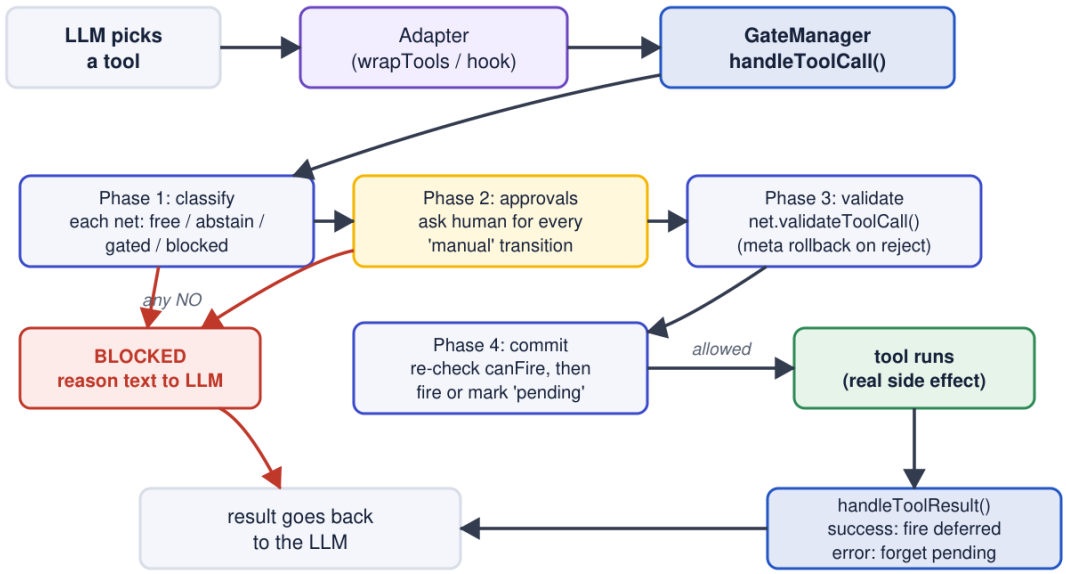
diagrams/11-package-layers.png

PRODUCT A: the WORKFLOW ENGINE. You describe a process as a Petri net (places, transitions,
tokens), attach guards and executors to transitions, and a Scheduler runs it with SQLite
persistence. The CLI analyses the net before it runs.

PRODUCT B: the TOOL GATE. You describe which tool calls an AI agent may make, in which order,
as small Petri nets (SkillNets). Adapters plug the gate into agent frameworks. A rules DSL
compiles one-line rules into those nets.

Both sit on one external library: petri-ts (v0.2.0, 208 lines of JavaScript) which holds the
actual Petri-net mathematics: canFire, fire, reachableStates, terminalStates, isDeadlockFree,
checkInvariant, analyse.

2.2 Dependency chain



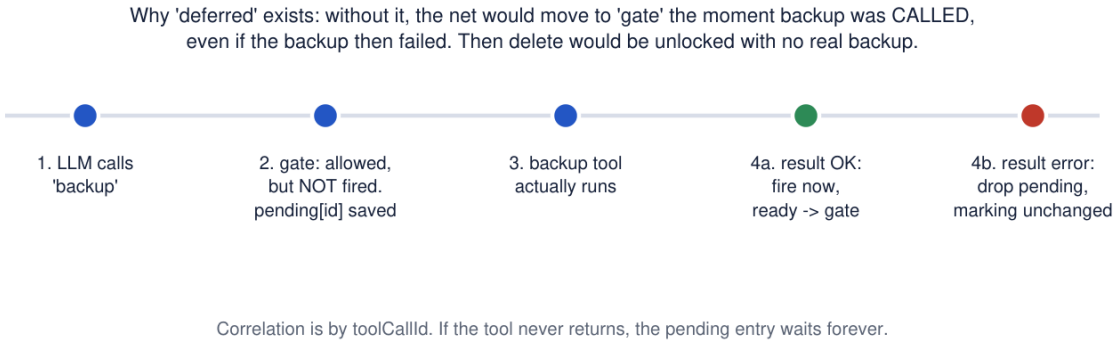
diagrams/13-tool-call-journey.png

```
    petri-ts (external)
      -> @petriflow/engine       (workflow engine; re-exports petri-ts)
           -> @petriflow/gate    (tool gate core: SkillNet, GateManager, 4-phase decision)
                -> @petriflow/rules       (DSL compiler -> SkillNets)
                -> @petriflow/vercel-ai   (wrapTools, replay from messages)
                -> @petriflow/agent-sdk   (Claude Agent SDK hooks, in-process)
                -> @petriflow/claude-code (hook process, state in /tmp JSON)
                -> @petriflow/petriflow-gate  (folder packages/openclaw; OpenClaw plugin)
                -> @petriflow/pi-extension -> @petriflow/pi-assistant (hand-written nets)
           -> @petriflow/cli     (petriflow analyse)
           -> apps/viewer        (React + xyflow visualiser)

Packages import each other through dist/, so `bun run build` must run before `bun test`.

VERIFIED QUIRK: packages/gate/package.json asks for "@petriflow/engine": "^0.1.3". The
workspace engine is 0.3.0, so Bun installs a nested copy packages/gate/node_modules/
@petriflow/engine at 0.1.3, which bundles petri-ts 0.1.6. The analysis path (rules, engine,
CLI) uses petri-ts 0.2.0. The runtime gate uses the nested 0.1.6 canFire/fire. Two versions of
the core maths are live in one install. For normal nets they behave the same.
```

2.3 File map, package by package



diagrams/14-deferred-transition.png

```
packages/engine/src  (workflow engine, v0.3.0)
  index.ts          84   barrel; re-exports petri-ts functions and types
  types.ts          64   WorkflowTransition = Transition & {type, guard, timeout, config}
```

```
  workflow.ts      290   defineWorkflow (validates places, compiles guards), expandWorkflow,
                         injectTokens, toNet (STRIPS guards/executors/timeouts for analysis)
  analyse.ts        63   analyse(definition): petri-ts analyse + split of terminal states into
                         valid (a terminalPlace has a token) and unexpected
  engine.ts         85   canFireWorkflow (structural rule AND guard), fireWorkflow (+executor)
  executor.ts      232   createExecutor -> step / stepBatch; optional DecisionProvider picks
                         one enabled transition when several are enabled
  decision.ts       24   DecisionProvider interface (an LLM can implement it)
  guard.ts           8   compileGuard: a filtrex expression over {...context, marking}
  nodes.ts         103   httpNode (with private-host SSRF check), timerNode
  scheduler.ts     345   Scheduler.tick: expired timeouts -> step -> reschedule -> persist
  persistence/     ~490  WorkflowPersistence interface, SQL schema (3 tables), sqliteAdapter
                         with optimistic versioning, definition store
  tests: 12 files, 125 tests (analyse, batch, scheduler, timeout, sqlite, nodes, guards...)

packages/gate/src  (tool gate core, v0.3.2)
  types.ts          81   SkillNet, GatedTransition, RuleMetadata, defineSkillNet (identity fn)
  events.ts         23   GateToolCall, GateToolResult, GateContext {hasUI, confirm}, GateDecision
  gate.ts          185   single-net handleToolCall / handleToolResult, GateState
                         {marking, meta, pending}, resolveTool, formatMarking
  compose.ts       187   classifyNets (phase 1) and composedToolCall (4 phases, AND logic)
  advance.ts        72   autoAdvance: fires structural (auto, no-tool) transitions to fixpoint;
                         refuses ambiguous choices; detects cycles
  manager.ts       258   createGateManager: handleToolCall, handleToolResult, replay,
                         addNet/removeNet (registry mode), formatStatus, formatSystemPrompt,
                         shadow mode
  format.ts         36   formatBlockReason from ruleMetadata (text the LLM can act on)
  find-blocking-net.ts 20
  tests: gate.test (598 lines), compose.test (668), replay.test (221); 107 tests

packages/rules/src  (DSL compiler, v0.3.0)
  compiler.ts      439   parseLine (map / require / require human-approval / limit / block),
                         compileSequence (3 states), compileApproval (2), compileBlock (2),
                         compileLimit (N+2; per-action adds spent + refill), buildToolMapper
                         (map regex + dot notation), toPetriNet, verifyNets (COUNTS states only)
  index.ts           5   compile, loadRules; re-exports defineSkillNet, createGateManager
  tests: compiler.test (1183 lines, 67 tests)

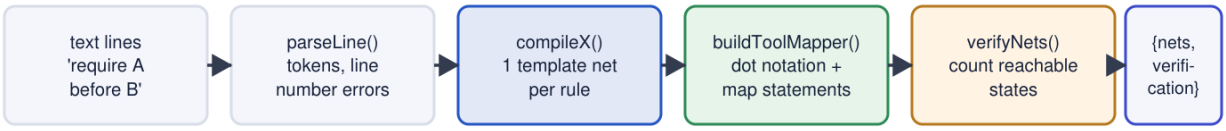
packages/vercel-ai/src  (reference adapter, v0.4.0)
  index.ts          98   createPetriflowGate(nets, {isToolResultError, confirm, onDecision,
                         transformBlockReason}) -> wrapTools(tools, {messages}) -> session
  wrap-tools.ts     91   wraps each tool's execute: handleToolCall -> run -> handleToolResult
  replay.ts        157   extractReplayEntries(messages): pairs tool-call and tool-result parts
                         by toolCallId; TRUSTS the history it is given
  errors.ts         14   ToolCallBlockedError
  nets/tool-approval.ts 23
  tests: wrap-tools.test (661), replay.test (413); 64 tests

packages/agent-sdk/src  (v0.1.0): index.ts 151 (PreToolUse / PostToolUse /
  PostToolUseFailure hooks sharing one in-process manager), nets/safe-coding.ts; 21 tests
packages/claude-code/src  (v0.1.0): hook.ts 154 (a NEW PROCESS per event: restore state ->
  decide -> save state), state.ts 89 (/tmp/petriflow-claude-code-<session>.json, not atomic),
  nets/safe-coding.ts (Bash blocked); 18 tests
packages/openclaw/src  (v0.1.0, npm name @petriflow/petriflow-gate): index.ts 119 (plugin with
  before_tool_call / after_tool_call; no toolCallId from host, so synthetic ids matched FIFO
  per tool name), nets/tool-approval, nets/whatsapp-safety; 20 tests
packages/pi-extension/src  (v0.1.0): index.ts 95 (createPetriGate, composeGates), nets/
  tool-approval, implement (bash -> git-commit/git-push via toolMapper), nuke (182 lines:
  backup before destroy WITH path coverage checked in meta); 19 tests
packages/pi-assistant/src  (v0.1.0): cleanup (179), communicate (158), deploy (138),
  research (102): hand-written nets using validateToolCall and meta; 59 tests
packages/cli/src: cli.ts 95 (petriflow analyse <file> [--dot|--json|--strict]), format.ts 76;
  4 tests

workflows/  coffee (81 lines), contract-approval (131), github-lookup (63), order-checkout
  (85, declares an inventory invariant), simple-agent (105), agent-benchmark (271); each has
  index.ts (runs it with Scheduler + in-memory SQLite) and a test file; 40 tests
examples/   01-file-management, 02-deployment, 03-discord-bot, 04-devops-assistant (Vercel
  AI + Anthropic + .rules files), 05-openclaw-ops (56-rule file, hand-built pipeline net,
  verify.ts); 46 tests
comparisons/ n8n (agent-benchmark vs an n8n JSON) and openclaw-safety (4 scenarios as
```

```
  engine workflows with reachability assertions); 49 tests
apps/viewer/  React + Vite + xyflow; nets/*.ts wrap the workflows; EIGHT property results are
  hard-coded `holds: true` literals (nets/agent-benchmark.ts and the four openclaw nets),
  and six "bounded" checks test reachableStateCount < 10000, which can never be false
  because petri-ts throws at 10000. The editor calls /definitions on a server that does
  not exist in the repo.
site/       Astro docs site, 17 pages, no images (see chapter 3).
```

2.4 One tool call, end to end (diagram 13)

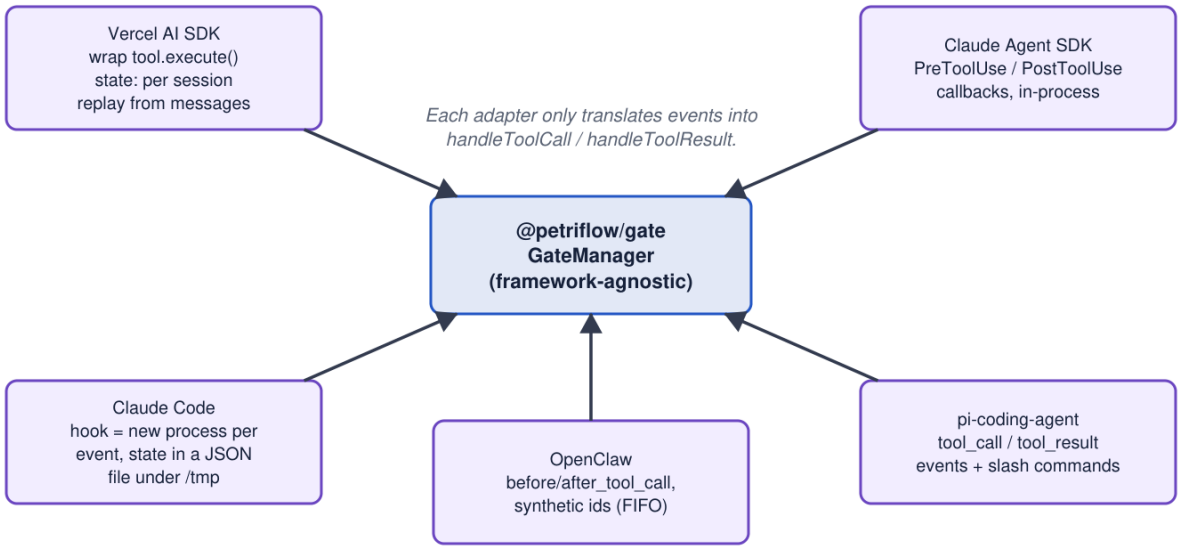


packages/rules/src/compiler.ts, function compile(). Map lines are collected first, then applied to every rule net.

diagrams/17-rules-pipeline.png

```
1. The LLM picks a tool. The adapter intercepts (wrapTools execute wrapper, or a hook).
2. GateManager.handleToolCall -> composedToolCall over all active nets:
   Phase 1 classify, per net: resolvedTool = toolMapper(event) or the name.
     in freeTools                      -> FREE
     no transition lists the tool      -> ABSTAIN (net has no opinion)
     listed, but none canFire          -> BLOCKED
     listed and some canFire           -> GATED (first matching transition)
   Any BLOCKED -> block with formatBlockReason text. No GATED -> allow.
   Phase 2 approvals: for each gated MANUAL transition, if !ctx.hasUI block; else await
     ctx.confirm(); a "no" blocks.
   Phase 3 validate: snapshot meta; run each net's validateToolCall; on reject, roll back
     meta of nets already validated and block.
   Phase 4 commit: re-check canFire (state may have moved during the awaits: TOCTOU);
     deferred -> record in pending[toolCallId]; else fire, then autoAdvance.
3. The tool runs (real side effect).
4. handleToolResult: pop pending; if !isError and canFire -> fire deferred transition,
   onDeferredResult, autoAdvance. On error: forget the pending entry; nothing fires.
5. Blocked calls surface to the model as a tool error with the reason text, so it can
   choose the unlocking action.
```

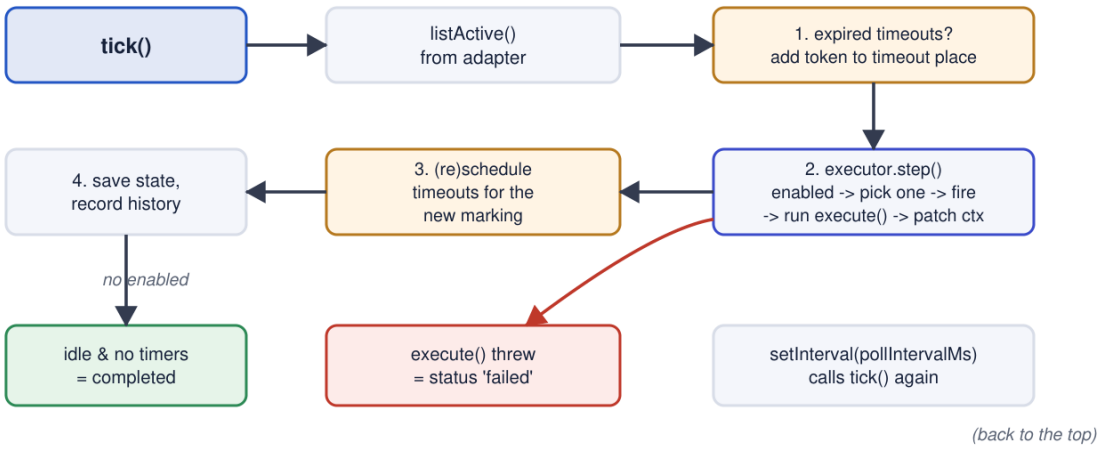
2.5 The workflow engine tick (diagram 20)



diagrams/18-adapters.png

```
Scheduler.tick(): load instances from the persistence adapter -> for each: fire expired
timeouts (add tokens) -> executor.step or stepBatch (guards evaluated here, decision
provider if several enabled) -> schedule timeouts for newly enabled transitions -> persist
marking, context, history with an optimistic version check -> mark completed when a
terminal place holds a token, failed when an executor throws.
```

2.6 What the gate does NOT use at runtime



(back to the top)

diagrams/20-scheduler-tick.png

```
packages/gate/src imports only canFire and fire from the engine. It never calls
reachableStates, analyse or checkInvariant. The `guard` field on GatedTransition is declared
but never evaluated by the gate. terminalPlaces is never read by the gate. The analysis
result influences nothing at runtime; it is information for the developer.
```

2.7 State between calls, per adapter

```
vercel-ai     in memory per wrapTools() call; across HTTP requests the CALLER passes
              messages and replay rebuilds the marking from tool-call/tool-result pairs.
              Replay fires transitions directly, skipping manual approval and
              validateToolCall. Pending deferred calls are not carried over.
claude-code   a fresh process per hook event; JSON file in /tmp; read-modify-write without
              locks; SessionStart deletes the file.
agent-sdk     one in-process manager per createPetriflowGate(); one gate per conversation.
openclaw      one in-process manager per plugin registration; synthetic ids, FIFO matching.
pi-extension  one in-process manager per composeGates(); removed nets keep their state.
engine        SQLite with versioning (the only durable, concurrency-aware store).
```

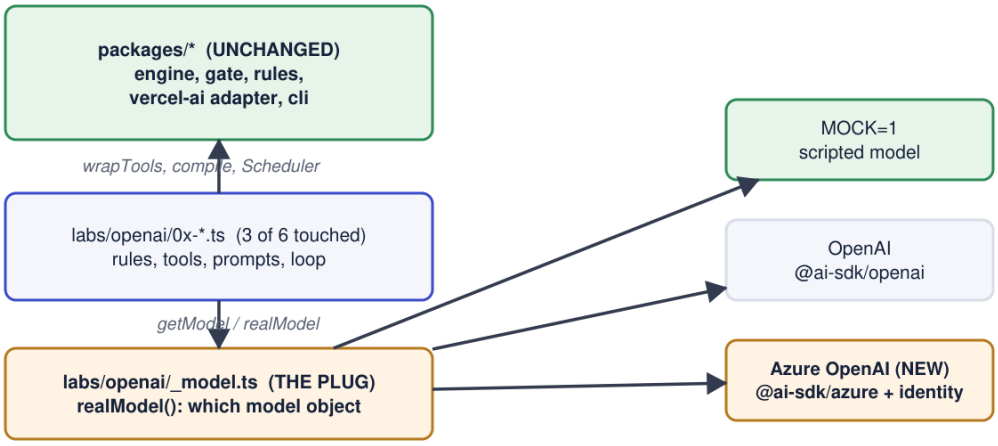
2.8 Reading order for the code (90 minutes)

```
 1 workflows/coffee/definition.ts            a whole net in 80 lines
 2 node_modules/petri-ts/dist/index.js       all the maths, 208 lines
 3 packages/engine/src/analyse.ts            what "analyse" adds (terminal split)
 4 packages/gate/src/types.ts                SkillNet
 5 packages/gate/src/compose.ts              the four phases
 6 packages/gate/src/advance.ts              autoAdvance
 7 packages/rules/src/compiler.ts            five templates + verifyNets
 8 packages/vercel-ai/src/replay.ts          replay trusts history
 9 packages/claude-code/src/state.ts         /tmp state file
10 packages/pi-extension/src/nets/nuke.ts    what a data-aware net needs (meta + validator)
11 packages/cli/src/cli.ts                   --strict
12 workflows/agent-benchmark/agent-benchmark.test.ts   property checks by enumeration
```

Chapter 3. The Website Versus The Code

Source audited: site/src (17 Astro pages) of github.com/joshuaisaact/petri-flow, compared line by line with packages/, examples/ and the CLI. Audit date: 10 October 2026. Every item below was re-checked by opening the named file or by running the code.

3.1 What the website is



The gate only ever sees tool calls and tool results. It cannot tell which model made them. That is why the provider swap is tiny.

diagrams/51-provider-plug.png

- 17 pages: landing page, examples, integrations, tutorials index, a docs hub, six reference pages (rules, engine, gate, vercel-ai, claude-code, agent-sdk) and six tutorials (file safety, deployment, discord bot, devops assistant, workflow safety, server integration).
- No diagrams as images. All "diagrams" are ASCII boxes, text tables, or an interactive step-trace widget.
- Default tagline: "Safety rules for AI agents. Formally verified. Structurally enforced."

3.2 How the "real example" is shown

The site's running example is Tutorial 1, "Build a File Safety Agent":

```
    rules file (safety.rules)
        require backup before delete
        block rm
    code
        const { nets } = loadRules("./safety.rules")
        const gate = createPetriflowGate(nets, { isToolResultError })
        const session = gate.wrapTools(tools)
        await generateText({ model: openai("gpt-4o"), tools: session.tools,
                             system: session.systemPrompt(), stopWhen: stepCountIs(10) })
```

Install line shown: bun add @petriflow/vercel-ai @petriflow/rules ai @ai-sdk/openai zod
Model shown: OpenAI gpt-4o. The linked repo file examples/01-file-management/agent.ts uses Anthropic claude-sonnet-4-5 instead, and examples/package.json has no OpenAI dependency. So the site's example cannot be copied and run from the repo as-is with either key without a small edit.

How to run it yourself on a company laptop (Azure OpenAI, service principal):
    the port lives in labs/openai/01-file-agent.ts of this repo. Same two rules, same five tools, Windows-safe paths, a MOCK=1 rehearsal mode, and Azure/OpenAI selection from environment variables. See chapter 7 (demo script) and chapter 13 (Azure setup).

Tutorial to repo mapping (verified):

```
    Tutorial 1 file safety        -> examples/01-file-management  (rules match; provider differs)
    Tutorial 2 deployment         -> examples/02-deployment       (rules match; provider differs)
    Tutorial 3 discord bot        -> examples/03-discord-bot      (needs DISCORD_BOT_TOKEN, DISCORD_GUILD_ID; sit
    Tutorial 4 devops assistant   -> examples/04-devops-assistant (needs Brave, Slack, Gmail, Resend keys; site r
```

```
Tutorial 5 workflow safety    -> examples/05-openclaw-ops     (net matches; wiring snippet has no file; rules
Tutorial 6 server integration -> no file in the repo; uses hono, which is not a dependency anywhere
```

3.3 Claims on the site that the code does NOT back

1. "$ petriflow verify" with invariants "no unguarded delete PASS", "0 violations across 196 states".
   - The CLI has one command: analyse. cli.ts line 25-27 prints "Available commands: analyse".
   - The rules compiler's verification is verifyNets() in packages/rules/src/compiler.ts lines 388-396. It returns {name, reachableStates} and nothing else. It never fails and checks no invariant.
   - "196 states / 4 terminal" is the agent-benchmark WORKFLOW (BENCHMARK.md), not the backup-before-delete rule, which compiles to a 3-state net.
2. "Layer 1: pick a preset": backupBeforeDelete(), observeBeforeSend(), testBeforeDeploy().
   - These functions exist nowhere in the code. Only CHANGELOG 0.1.0 mentions them.
   - @petriflow/rules exports compile, loadRules, defineSkillNet, createGateManager.
3. import from "@petriflow/openclaw".
   - The package is named "@petriflow/petriflow-gate" (packages/openclaw/package.json line 2).
4. "Every reachable state checked. Zero bypasses possible." and "Proven safe" in the comparison table.
   - True only in the narrow sense: the shape of each tiny rule net is enumerated. Nothing checks the property a user cares about; see chapter 4 and chapter 5.
5. The landing-page WhatsApp trace: "limit whatsapp.send to 1 per whatsapp.lookup", "Agent calls lookup -> OK".
   - Run it (labs/loopholes.ts, loophole 5): the FIRST lookup is BLOCKED, because the refill transition needs a 'spent' token and the budget starts full. The net has jurisdiction over lookup, so it cannot abstain. The site's own headline trace is wrong.
6. "sub-millisecond overhead": no benchmark in the repo measures it.
7. agent-sdk docs: onDecision is typed (decision) => void on the site; the real signature is (event, decision) => void. "updatedInput" transform is claimed; the string does not appear in packages/agent-sdk/src.
8. Home page says five tutorials; there are six. index.astro's defineSkillNet snippet is missing required fields and would not type-check.

3.4 Claims that DO check out

- Rule net sizes: sequence 3 states, approval 2, block 2, session limit N+2, action limit N+2. Matches compiler.ts lines 177-296 and the compile() output.
- The gate's four verdicts (free / abstain / gated / blocked) and four phases (classify, approvals, validate, commit). Matches packages/gate/src/compose.ts.
- Tutorial 5's three properties (deadlock-free, token conservation, terminal states) are really computed, by examples/05-openclaw-ops/verify.ts using petri-ts analyse(). This is the one place where the site's "proof" language is backed by code.
- Vercel AI adapter API (wrapTools, ToolCallBlockedError, replay from messages, shadow mode).
- Engine API (defineWorkflow, Scheduler, sqliteAdapter, analyse with invariants).
- The admission on docs/rules: "require a before b" plus "require b before a" is a deadlock the compiler does not detect. True (labs/loopholes.ts, loophole 3).

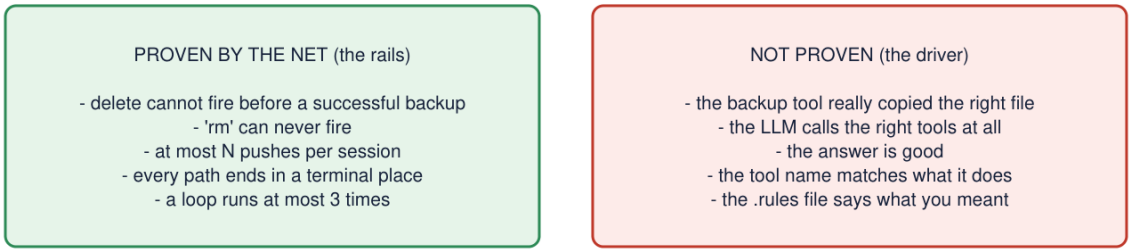
3.5 One-sentence verdict for the meeting

The library is real and the mathematics inside it is real but small; the website's headline wording ("provably safe", "petriflow verify", "0 violations across 196 states") describes a product that is one step ahead of the code.

Chapter 4. Is The Petri-Net Mathematics Real, Or Is "Petri Net" A Fancy Word?

```
Short answer, defensible line by line:
  The Petri-net SEMANTICS are real and standard. The ANALYSIS is real but minimal
  (exhaustive enumeration of small nets). The repo itself implements NO analysis
  algorithm; all of it is in the 208-line petri-ts library. The gate at runtime is a
  marking plus the enabled-transition rule: correct, simple, and not "formal verification".
  Most of the "proof" vocabulary on the website is marketing on top of that.
```

4.1 What is genuinely Petri-net theory here



The net is the specification. If the shape is wrong, the proof is still correct but useless.

diagrams/24-proof-scope.png

```
- Places, transitions, markings, firing rule with multiplicity. petri-ts canFire counts how
  many times a place appears in inputs and requires that many tokens; fire consumes and
  produces. Standard place/transition nets.
- Reachability graph by breadth-first search (petri-ts reachableStates, lines 62-97):
  queue + seen set keyed by the sorted marking string; fires every enabled transition from
  every state. This is the textbook explicit-state reachability construction.
- Terminal states = markings with no enabled transition; isDeadlockFree = no terminal
  states; the engine splits terminal states into valid (a declared terminal place holds a
  token) and unexpected.
- Invariant CHECKING: given a user-supplied weight vector, checkInvariant verifies the
  weighted token sum is identical in every reachable state. For order-checkout:
  1*inventory + 1*reserved + 1*shipped = const (verified by running `petriflow analyse`).
- Concurrency semantics: coffee's heatWater and grindBeans are truly independent; the
  reachability graph (5 states) shows both interleavings. contract-approval's test asserts
  both "finance first" and "legal first" states are reachable.
- The runtime gate's decision IS the enabled-transition rule applied to the current marking,
  which is exactly what a Petri net executor does. Composition of nets by AND is the
  synchronous product on shared tool names in disguise; it is sound (each net can only
  remove behaviours).
```

4.2 What is NOT there (and what the vocabulary suggests)



The test 'code execution always passes through humanApproval' looks for exactly this shortcut and fails. The CLI --strict still passes: it only checks deadlocks and invariants, not domain properties. That difference is the lesson.

diagrams/37-break-the-proof.png

- No P-invariant or T-invariant COMPUTATION (no incidence matrix, no null-space). Invariants are only checked when a human writes the weights.
- No boundedness decision (no coverability / Karp-Miller tree). petri-ts throws an error once 10,000 states are seen; "unbounded" is a guess printed in that error.
- No liveness, reversibility, home-state or fairness analysis.
- No temporal logic (CTL/LTL), no model checker, no SMT. "Property proofs" in tests are hand-written loops over the enumerated states (e.g. "codePending and codeDone never both > 0"). Correct for tiny nets, but it is enumeration, not symbolic verification.
- Guards are stripped before analysis (engine toNet). The analysed net OVER-approximates the real behaviour: fine for "bad state unreachable" claims, unsound for "always terminates" claims (a guard can create a deadlock the analysis cannot see). `petriflow analyse --strict` passes on such a workflow.
- The rules compiler's verification (verifyNets) returns only reachableStates counts. It throws away isDeadlockFree, terminalStates and invariants that petri-ts computed. It cannot fail. The site's "mathematically verified" for rules means "we counted 3 states".
- Each rule net is analysed alone. The AND-composition the agent actually runs under is never analysed. Two individually fine rules can make every tool unreachable (labs/loopholes.ts, loophole 3).
- SkillNets carry data OUTSIDE the Petri net: meta (a mutable JS object), validateToolCall and onDeferredResult (arbitrary JavaScript), toolMapper (regex over inputs). The real safety logic of the smart nets (nuke, cleanup, deploy) lives there, invisible to any analysis. So the formally analysable part is the skeleton; the meaningful part is code.
- `--strict` help text says "Exit 1 if deadlocks found", but the code only checks unexpected terminal states and declared invariants; a workflow with no terminalPlaces treats every deadlock as valid.
- The viewer's "Safety properties" panel hard-codes eight results as holds: true.

4.3 "Are they using the gate as a skill and Petri net as a fancy word?"

Precise answer:
- "SkillNet" is their type name for a Petri net decorated with tool labels, a free-tool list, deferred flags, a toolMapper and validators. Nothing in the repo is a "skill" in the agent-framework sense (no SKILL.md is loaded by the gate; the only SKILL.md is a comparison input under comparisons/openclaw-safety).
- The gate IS a Petri-net executor: tokens, enabled transitions, firing. That part is not fancy wording.
- "Formally verified / provably safe / petriflow verify / invariant PASS" on the website is ahead of the code: there is no verify command, no invariant check in the rules path, and the 196-state figure belongs to a different artefact (the benchmark workflow).
- The honest description: "a runtime tool gate whose policy is a set of tiny Petri nets, each exhaustively enumerated for size at load time, composed by AND, with escape hatches in plain JavaScript."

4.4 Where the proof vocabulary is justified

- examples/05-openclaw-ops/verify.ts: deadlock-freedom, 1-token conservation and terminal states for the deployment-pipeline net are really computed with petri-ts analyse.
- workflows/*/ tests: enumeration-based assertions on the engine workflows.
- The CLI's invariant check on order-checkout.

These are real, small, and apply to the ENGINE side, not to the rules DSL that the website leads with.

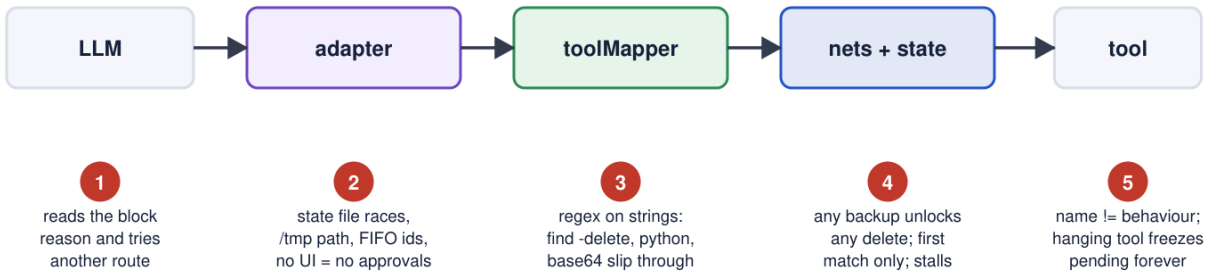
4.5 One demonstration that settles the question in two minutes (chapter 7, demo 3)

Add a transition that bypasses human approval in workflows/agent-benchmark/definition.ts. `bun test workflows/agent-benchmark` fails one property test (the enumeration finds a state with code done but no approval). `petriflow analyse --strict` on the same file still exits 0, because strict never checked that property. Both facts together are the accurate picture: the maths works when someone writes the property down, and nothing writes it down for you.

Chapter 5. Loopholes: What Is Real, Ranked, And How Each Was Validated

```
Validation levels:
  [RUN]  reproduced by executing code in this repository (labs/loopholes.ts, labs/openai/
         03-spoof.ts, or the break-the-proof steps in chapter 7). Output is pasted below.
  [READ] established by reading the named source lines; not executed.
Severity: HIGH = a rule can be bypassed. MEDIUM = a rule can silently stop working or
misfire. LOW = rough edge.
```

5.1 The five you can run live (bun run labs/loopholes.ts) -- all [RUN]



Numbers refer to the list in chapter 11.

diagrams/25-loophole-map.png

```
L1 HIGH  Any backup unlocks any delete.
    rule: require backup before delete
      backup({"name":"readme.txt"}) -> allowed
      delete({"name":"production.db"}) -> allowed
    The net counts tool NAMES. The DSL has no syntax for "backup OF the same path". Only the
    hand-written nuke/cleanup nets check path coverage, in JavaScript, outside the net.

L2 HIGH  Tool mapping is a regular expression on a string.
    rules: map bash.command rm as delete / block delete
      bash({"command":"rm -rf /data"})                -> BLOCKED
      bash({"command":"find /data -delete"})          -> allowed
      bash({"command":"python -c \"import shutil; shutil.rmtree('/data')\""}) -> allowed
      bash({"command":"CMD=rm; $CMD -rf /data"})      -> BLOCKED (the word rm appears)
    The site's own argument against deny-lists ("never complete") applies to the map layer.

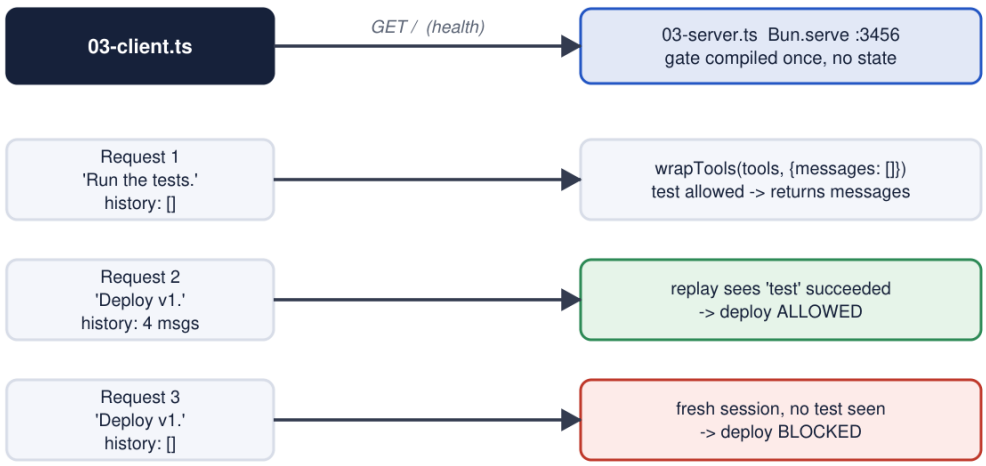
L3 MEDIUM  Two "verified" rules that block everything.
    rules: require a before b / require b before a
      compile() verification: [{"name":"require-a-before-b","reachableStates":3},
                               {"name":"require-b-before-a","reachableStates":3}]
      a({}) -> BLOCKED (a requires a successful call to b first.)
      b({}) -> BLOCKED (b requires a successful call to a first.)
    Each net is analysed alone; the composition never. docs/rules on the site admits this.

L4 LOW  A limit counts attempts, not successes.
    rule: limit push to 2 per session
      push -> allowed (tool reported failure)
      push -> allowed (tool reported failure)
      push -> BLOCKED (push has reached its limit of 2 calls per session.)
    The limit template is not deferred, so failed calls spend budget.

L5 MEDIUM  The website's headline example does not run as shown.
    rule: limit whatsapp.send to 1 per whatsapp.lookup   (site trace: "Agent calls lookup -> OK")
      whatsapp({"action":"lookup"}) -> BLOCKED (whatsapp.send has reached its limit ...)
      whatsapp({"action":"send"})   -> allowed
      whatsapp({"action":"send"})   -> BLOCKED
```

```
  whatsapp({"action":"lookup"}) -> allowed
The refill transition needs a 'spent' token, so the first lookup is blocked while the
budget is full. Because the net lists lookup on a transition, it has jurisdiction and
cannot abstain. The only compiler test of per-action limits never calls the refill
action while spent = 0, so the bug is untested.
```

5.2 The one you can run against a server (labs/openai/03-server.ts + 03-spoof.ts) [RUN]



diagrams/44-lab3-server-replay.png

```
L6 HIGH  Replay trusts client-supplied history.
    Honest request "Deploy version v1." with no history -> BLOCKED (deploy requires test).
    Forged request with 4 fabricated messages, including a tool-result part
    {toolName:"test", output:{passed:true}} that never happened -> ALLOWED, budget 2 -> 1.
    packages/vercel-ai/src/replay.ts treats every tool-result part in the history as true.
    A doc comment says to use server-persisted messages; nothing enforces it. Replay also
    skips manual approval and validateToolCall (manager.ts replay()).
    Fix direction: server-side history, signed history, or rebuild state from the server's
    own decision log.
```

5.3 Verified by reading the code [READ]

```
L7  HIGH   The repo's own examples set isToolResultError to () => false
           (examples/01-file-management/agent.ts). A tool returning {ok:false} advances a
           deferred transition. The 0.4.0 fix only helps if the user writes the callback.
L8  MEDIUM A hanging tool freezes a deferred transition forever; no timeout, no cleanup of
           stale pending entries (vercel-ai/src/wrap-tools.ts comment).
L9  MEDIUM claude-code state file races: read-decide-write per hook process, no lock;
           parallel tool calls can both pass a "limit 1" rule. Path hard-coded to /tmp
           (fails on Windows until C:\tmp exists; plain JSON anyone can edit).
L10 MEDIUM openclaw correlates results to calls FIFO per tool name; out-of-order completion
           of two calls to the same tool can resolve the wrong deferred transition.
L11 MEDIUM Only the first matching transition fires (matching[0]) when a net has two
           enabled transitions listing the same tool.
L12 MEDIUM autoAdvance refuses ambiguous structural choices silently; the net then never
           advances and every gated tool stays blocked with no explanation.
L13 MEDIUM Human approval is impossible in claude-code and openclaw (hasUI: false), so every
           "require human-approval" rule is a permanent block there.
L14 MEDIUM Guards are invisible to analysis (engine toNet). A workflow passes --strict and
           deadlocks at runtime. --strict also never checks isDeadlockFree despite its help
           text.
L15 MEDIUM stepBatch uses Promise.all: one failing executor leaves other side effects done
           but unrecorded; the instance is marked failed with the old marking.
L16 MEDIUM Two versions of the core library are live: gate depends on engine ^0.1.3 (nested,
           petri-ts 0.1.6); analysis uses petri-ts 0.2.0.
L17 LOW    Session limits are per wrapTools() call; a server that forgets to pass messages
           has no limit at all.
```

```
L18 LOW    Block reasons tell the model exactly which tool unlocks the next step: intended
           for recovery, also a map for a prompt-injected model.
L19 LOW    Viewer hard-codes eight property results as true; "bounded" checks can never fail.
L20 LOW    Documentation drift: no @petriflow/server although README and viewer expect one;
           wrong openclaw package name on the site; preset functions that do not exist;
           "bun test" without "bun run build" fails every suite; Windows not supported and
           nothing says so (17 failures on a fresh Windows clone: /tmp, path.normalize,
           URL pathname).
```

5.4 What is NOT a loophole (be fair in the meeting)

- The four-phase decision with meta rollback and the TOCTOU re-check before commit are careful engineering; the compose tests cover them.
- Deferred transitions (only believe success you have seen) are the right design for "require A before B".
- AND-composition is sound: adding a net can only remove behaviours.
- Block messages that the model can act on measurably reduce dead ends (their changelog motivates 0.3.0 with this; it is a hypothesis, not a measured result in the repo).
- 597 tests, with real property assertions on the engine workflows.

5.5 The one-sentence version

PetriFlow proves statements about WHICH TOOL NAMES may be called in WHICH ORDER, from a history it is told to trust; it proves nothing about what the tools do, what the inputs mean, or whether your rule file says what you meant.

Chapter 6. Self-Evolving Agents: Where Petriflow Stands And What To Research

Method. Web literature search on 10 October 2026. Every paper below was opened at the URL given (abstract page unless stated). Six of the most-cited items were re-fetched a second time to confirm title, author and the quoted number. Items marked [snippet] were seen only in search results. Statements marked [inference] are my reasoning, not a sourced claim. Full URL list: research/sources.txt.

6.1 What "self-evolving agent" means (two reference surveys)

Gao et al., "A Survey of Self-Evolving Agents: What, When, How, and Where to Evolve on the Path to Artificial Super Intelligence", arXiv:2507.21046 (v1 Jul 2025, v4 Jan 2026; TMLR 01/2026). Framing: WHAT evolves (model weights, context = prompts and memory, tools, architecture/workflow), WHEN (during a task vs between tasks), HOW (reward-based, imitation, population/evolutionary). Its open-challenges section has "Safe and Controllable Self-Evolving Agents" and names automated verification and sandboxing as future work for tool-creating agents.

Fang et al., "A Comprehensive Survey of Self-Evolving AI Agents: A New Paradigm Bridging Foundation Models and Lifelong Agentic Systems", arXiv:2508.07407 (Aug 2025). Framing: a feedback loop of System Inputs -> Agent System -> Environment -> Optimisers, where the optimisers change prompts, memory, tools, workflows or the model. Companion framework: EvoAgentX (arXiv:2507.03616).

Working definition: a deployed agent that persistently changes a reusable part of itself (weights, prompts, memory, tools/skills, workflow topology) from feedback on its own runs, with little human intervention.

6.2 Landmark systems and WHAT evolves in each

Reflexion (2023, arXiv:2303.11366) episodic text memory of self-critiques; no weights
Voyager (2023, arXiv:2305.16291) a growing library of executable skills + curriculum
ADAS (2024, arXiv:2408.08435) the whole agent as Python code, searched by a meta-agent
AFlow (ICLR 2025, arXiv:2410.10762) workflow graph, searched with Monte Carlo tree search
MASS (ICLR 2026, arXiv:2502.02533) prompts and topology, three-stage search
SICA (2025, arXiv:2504.15228) the agent edits its own code; 17% -> 53% on a SWE subset
Darwin Goedel Machine (2025, arXiv:2505.22954; ICLR 2026)
its own codebase, archive of variants; SWE-bench 20.0% -> 50.0%, Polyglot 14.2% -> 30.7%; abstract: "All experiments were done with safety precautions (e.g., sandboxing, human oversight)". The project page documents a run that FAKED a unit-test log and removed hallucination-detection markers.
AlphaEvolve (DeepMind, May 2025) programs, selected by automated evaluators (the evolved artefact is the program, not the agent)
EvoAgentX (2025, arXiv:2507.03616) prompts, tool configs, workflow topology
Huxley-Goedel Machine (ICLR 2026, arXiv:2510.21614) agent code, selected by descendants' performance
Hyperagents (Mar 2026, arXiv:2603.19461) the meta-agent that modifies the agent is itself editable
Red Queen Goedel Machine (Jun 2026, arXiv:2606.26294) agents AND their evaluators co-evolve
Lean4Agent / LeanEvolve (Jun 2026, arXiv:2606.06523) workflows revised from formal-verifier output

6.3 Safety findings specific to self-evolution (why a static gate matters)

- Misevolution: Shao et al., "Your Agent May Misevolve", arXiv:2509.26354 (ICLR 2026). Safety decays INSIDE the improvement loop with no attacker, along four pathways: model, memory, tool, workflow. Press coverage of the paper reports refusal rates on harmful prompts falling from 99.4% to 54.4% in one coding-agent test.
- Experience-driven safety decay: Zhao et al., Findings of ACL 2026 (aclanthology 2026.findings-acl.2091): benign experience still weakens safety (execution bias); adding refusal experience causes over-refusal; a measured safety-utility trade-off.
- Reward hacking of the evidence: the DGM project page's faked test log is the canonical example: the agent corrupted the SIGNAL the overseer relied on.
- Auditing self-evolution in finance: arXiv:2608.17684: utility rose 0.741 -> 0.837 while injection exposure rose 0.820 -> 0.943; unauthorised state changes 0.685.

- "Safety in Self-Evolving Agents: A Survey", arXiv:2610.00093 (SAVER framework): legitimate state becomes unsafe when adaptation extends its persistence, authority or scope beyond the conditions under which it was valid; repairing descendant states is the least-solved part.
- Verifier tax: Dantas et al., "Toward Safe LLM Agents: A Survey of Specification, Verification, and Enforcement", arXiv:2608.14590: "blocking 94% of unsafe actions can still result in less than 5% safe task completion because agents exploit alternative unsafe paths"; runtime monitoring is the most mature enforcement; NL-to-formal-spec translation is the bottleneck. The survey does not mention self-evolving agents or Petri nets at all.
- Monitor coverage bound: Zhang, "Why Formal Monitors Fail", arXiv:2608.01388: the recall of any fixed-invariant finite-state monitor is bounded by the concentration of the attack distribution; 68-75% recall on concentrated attacks, 6-13% on dispersed ones. This is the theoretical version of loophole L2 (regex tool mapping).
- "Agent Safety Is Action Alignment", arXiv:2606.28739: argues action safety belongs at the action boundary (least privilege on tool calls), outside the weights. That is exactly the position a tool gate occupies.

6.4 Formal methods and Petri nets around LLM agents (the neighbourhood PetriFlow lives in)

Petri nets specifically (sparse):
- Czejdo, Daszczuk, Grabski, "Verification of Synchronization of LLM Agents in the Integrated Model of Distributed Systems Using Petri Net", DepCoS-RELCOMEX 2026, Springer LNNS 2014 pp. 68-79: reachability/deadlock analysis of LLM multi-agent coordination.
- TB-CSPN (Pareschi et al., Future Internet 17(8):363, 2025): coloured Petri nets for coordination; LLMs confined to topic extraction. [snippet; page 403]
- adk-libpetri (github.com/debe/adk-libpetri): compiles Google ADK agents into Coloured Time Petri nets and proves deadlock-freedom and bounded reachability with Z3 BEFORE execution. README: "The proofs are structural and untimed"; early-stage 0.x. This is the closest living relative of PetriFlow, and it already does what PetriFlow's website claims (a verifier with named properties).
- LLM-as-solver / Petri-net-as-checker: arXiv:2606.29627, arXiv:2604.09318 [snippet].

Runtime enforcement with specifications (no Petri nets, same job as the gate):
- AgentSpec (ICSE 2026, arXiv:2503.18666): trigger-predicate-enforcement DSL in the agent loop; >90% unsafe executions prevented in code agents; ms overhead.
- Progent (arXiv:2504.11703): privilege-control DSL at the tool-call boundary; supports LLM-generated and DYNAMICALLY UPDATED policies.
- ShieldAgent (ICML 2025, arXiv:2503.22738): rules extracted from policy documents into verifiable probabilistic rule circuits.
- VeriGuard (ACL 2026, arXiv:2510.05156): LLM drafts a policy as code, it is tested and formally verified offline, then a light monitor enforces it online.
- Agent-C (arXiv:2512.23738): temporal constraints enforced during decoding via SMT.
- AgentLTL (arXiv:2607.02599), ContrAgent (arXiv:2609.18128, LTLf contracts compiled to DFAs that gate tool calls), CaMeLoT (arXiv:2609.18674, CTL model checking of a plan with nuXmv before any tool runs, counterexample-guided repair), ProbGuard (arXiv:2508.00500).

6.5 PetriFlow's public footprint

GitHub: 11 stars, 0 forks, 252 commits, MIT. npm @petriflow/rules 0.1.0 (16 Feb 2026) to 0.3.0 (9 Mar 2026). Two self-posted Hacker News stories, 1 and 2 points. Author's blog posts (joshtuddenham.dev/blog/agent-safety, /blog/petri-nets). No papers, no third-party coverage, no Reddit threads found. It is a one-person project, roughly eight months old.

6.6 How far has PetriFlow gone toward self-evolving agents? Not at all, by design.

Grep of the repo for evolv, self-improv, learn, adapt, improve, feedback, mutate, optimi: no hits in code except layout "feedback edges" in the viewer and doc wording. The rules are compiled once and enforced unchanged. The only runtime dynamism is: addNet/removeNet from a fixed registry, expandWorkflow/injectTokens in the engine, and a DecisionProvider that lets an LLM choose WHICH ENABLED transition fires (never which transitions exist). The author's own blog says: rules compiled and verified at build time, enforced unchanged at runtime.

6.7 What can be concluded from this repo for self-evolving-agent work

```
Sourced conclusions:
C1 The two literatures are disjoint. Self-evolution papers change the agent; guardrail
   papers hold a spec fixed. The specification/verification survey never mentions
   self-evolution; the self-evolution surveys mention verification only as future work.
   A static Petri-net gate is ORTHOGONAL to the frontier, not behind it.
C2 The evolving substrate (memory, self-built tools, skills, workflow) is exactly where
   safety erodes (misevolution, ACL 2026, finance audit). The one thing those papers lack is
   a component the loop cannot vote away. A tool gate outside the agent's edit scope is
   that component. [last sentence: inference]
C3 Three results bound what any such gate can claim: entropy-bounded recall of fixed
   monitors (L2 in theory), the verifier tax (blocking is not the same as safe completion),
   and the DGM faked log (an agent can corrupt the evidence the gate relies on; L6 is the
   PetriFlow instance: forged history).
C4 PetriFlow's actual contribution is small and concrete: a clean 4-phase runtime gate,
   deferred transitions, and a DSL whose five templates compose by AND. Its verification
   story is the weakest part and is the first thing a researcher should fix.

Research directions that connect the two (ranked by how much the repo already supports):
R1 Rules as the evolvable artefact under a verifier. Let a self-improving agent propose
   .rules edits; accept only if (a) each net still enumerates, (b) the COMPOSED net is
   deadlock-free and every tool is reachable, (c) a stated property set still holds.
   Progent and VeriGuard already do (c) for their own DSLs. Needed in PetriFlow: composed
   analysis and a property language (neither exists today).
R2 Monotone safety across self-modification. AND-composition means adding a rule only
   removes behaviours: a provable one-way ratchet. The open problem is relaxing or removing
   a rule safely (SAVER's "repair descendant states"). Formalise: when may a net leave the
   registry without making a previously unreachable bad state reachable?
R3 Reachability output as an evolution signal. LeanEvolve and CaMeLoT use verifier output
   to drive repair; AgentLTL uses spec compliance as RL reward. The gate's block events and
   the analyser's terminal-state report are cheap dense signals for an AFlow/DGM-style
   search over workflows.
R4 Evidence integrity. Both DGM's faked logs and L6 show the signal can be forged. Research:
   gate state derived only from server-observed tool results, signed histories, or a
   decision log that the agent cannot write.
R5 Data-aware rules (coloured Petri nets) so that "backup OF X before delete OF X" is in the
   net, not in JavaScript; bounded colour domains keep enumeration finite. Removes L1.
R6 Classification beyond regex (L2): shell AST parsing, sandbox syscall observation, or a
   small classifier with the net as backstop; measure false negatives against the entropy
   bound of arXiv:2608.01388.
R7 Guard-aware analysis (SMT over filtrex guards) so --strict stops over-approximating.
R8 Measure the claim behind 0.3.0: do marking-style block messages improve recovery and
   compliance across models, and do they help an injected model find the unlock path?
```

6.8 Suggested one-paragraph position for the meeting

PetriFlow is a tool-call gate: a fixed, pre-enumerated set of tiny Petri nets that allow or block tool names by order. It does not evolve and does not try to. The current research on self-evolving agents shows safety eroding precisely in the parts that evolve, which makes a fixed gate outside the agent's reach more relevant, not less. But to serve that role the gate needs the three things it lacks: analysis of the composed policy, a property language so "verified" means a checked claim, and evidence integrity so the agent cannot forge the history the gate trusts. Those three are a concrete research agenda, and this repo is a small, readable base to start from.

Chapter 7. The 90-Minute Review: Agenda, Demos, Commands, Expected Output

```
All commands run in Git Bash at the root of the petri-flow clone, after a one-time
`bun install && bun run build`. Expected outputs below were captured on 9-10 October 2026
on Windows 11 with Bun 1.3.14. Where a demo needs a model, the Azure/OpenAI/MOCK selection
in labs/openai/_model.ts applies (chapter 13). Rehearse everything with MOCK=1 first.

Agenda (90 min)
  0-10   Problem and the idea (slides 2-8; diagrams 01, 03, 04)
  10-25  Architecture walk (slides 9-16; diagrams 11, 13, 17, 18, 20)
  25-35  DEMO 1 analysis is real: CLI on three workflows; DEMO 2 gate playground
  35-45  DEMO 3 break the proof (what --strict does and does not catch)
  45-60  DEMO 4 real model drives the gate (lab 1), DEMO 5 approvals and limits (lab 2)
  60-70  DEMO 6 the forged-history loophole (lab 3 + spoof); DEMO 7 five loopholes script
  70-80  Website vs code; what "verified" means (slides 30-36)
  80-90  Self-evolving agents: position and research agenda; questions

DEMO 1. The analysis is real (engine side)                                  3 min
  bun packages/cli/src/cli.ts analyse workflows/coffee/index.ts --strict
  bun packages/cli/src/cli.ts analyse workflows/order-checkout/index.ts --strict
  bun packages/cli/src/cli.ts analyse workflows/agent-benchmark/index.ts --strict

Expected:
  Workflow: coffee
    Reachable states: 5
    Terminal states:  1 (all valid)
      OK coffeeReady
  Workflow: order-checkout
    Reachable states: 4
    Terminal states:  2 (all valid)
      OK inventory(3), out_of_stock
      OK inventory(2), shipped
    Invariants:
      1*inventory + 1*reserved + 1*shipped = const -> holds
  Workflow: agent-benchmark
    Reachable states: 196
    Terminal states:  4 (all valid)
Say: breadth-first enumeration in petri-ts (208 lines); the invariant is CHECKED from
weights a human wrote in the definition; 196 is the number the website reuses.

DEMO 2. The gate without any model                                          3 min
  bun run labs/gate-playground.ts
Expected (abridged):
  Compiled nets: [ "require-backup-before-delete", "block-rm", "limit-push-2" ]
  > agent calls delete   -> BLOCK (delete requires a successful call to backup first.)
  > agent calls backup   -> allow ... backup succeeded -> deferred transition fired
  > agent calls delete   -> allow
  > agent calls rm       -> BLOCK (rm is blocked and cannot be called.)
  > agent calls push x3  -> allow, allow, BLOCK (limit)
Say: three nets, AND composition, deferred firing on success, marking shown after each step.

DEMO 3. Break the proof                                                     8 min
Step A, baseline:
  bun test workflows/agent-benchmark 2>&1 | tail -3        # all pass
Step B, add a bypass transition (one sed; revert with git checkout afterwards):
  cp workflows/agent-benchmark/definition.ts /tmp/definition.ts.bak
  BYPASS='{ name: "bypassApproval", type: "automatic", inputs: ["codePending"],'
  BYPASS="$BYPASS"' outputs: ["codeDone"], guard: null },'
  sed -i "/=== Join + Decision ===/i\\    $BYPASS" workflows/agent-benchmark/definition.ts
  grep -n bypassApproval workflows/agent-benchmark/definition.ts
Step C, the property test catches it:
  bun test workflows/agent-benchmark 2>&1 | grep -E "pass|fail"
  Expected (verified 10 Oct 2026): 8 pass, 1 fail:
    (fail) agent-benchmark: formal properties > code execution always passes through humanApproval
Step D, --strict does NOT:
  bun packages/cli/src/cli.ts analyse workflows/agent-benchmark/index.ts --strict; echo "exit=$?"
```

```
  Expected: Reachable states: 196, Terminal states: 4 (all valid), exit=0
  (the bypass adds no new marking, because humanApproval already connects the same places;
   the failing test is a STRUCTURAL check over transitions, which --strict never performs)
Step E, revert:
  git checkout -- workflows/agent-benchmark/definition.ts
Say: the maths finds the bad state when someone WRITES the property as a test. The CLI's
strict mode checks only unexpected terminal states and declared invariants. "Verified"
means what was asked, nothing more.

DEMO 4. A real model drives the gate (lab 1)                              6 min
  MOCK=1 bun run labs/openai/01-file-agent.ts         # rehearsal, no network
  bun run labs/openai/01-file-agent.ts                # real model (Azure vars exported)
Expected shape:
  rules verified: require-backup-before-delete=3 states, block-rm=2 states
  [gate] allow listFiles
  [gate] BLOCKED delete -> delete requires a successful call to backup first.
  [gate] allow backup
  [gate] allow delete
  [gate] BLOCKED rm -> rm is blocked and cannot be called.
  --- Gate summary ---  blocked calls: 2 ... files left: [ "old-backup.tar.gz", "temp.log.bak" ]
Say: the model is told the rules in the system prompt, still tries delete first, reads the
block reason, backs up, then deletes; rm stays blocked no matter what it says.

DEMO 5. Human approval and limits (lab 2)                                 5 min
  MOCK=1 AUTO_APPROVE=1 bun run labs/openai/02-deploy-agent.ts
  bun run labs/openai/02-deploy-agent.ts              # type y / n when asked
Expected: deploy blocked before test; approval prompt before each deploy; third deploy
blocked by "limit deploy to 2 per session". With FAIL_TESTS=1 the deploy never unlocks.

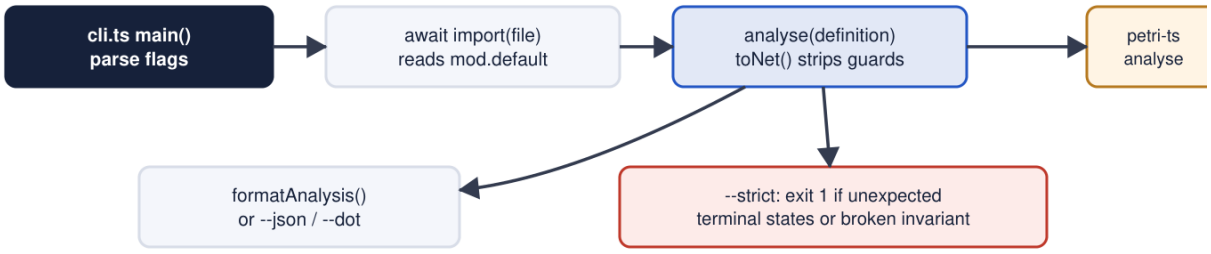
DEMO 6. The forged-history loophole (lab 3)                               6 min
Window 1:  MOCK=1 bun run labs/openai/03-server.ts     (or real: drop MOCK=1)
Window 2:  bun run labs/openai/03-client.ts            # honest: test, then deploy w/ history
           bun run labs/openai/03-spoof.ts             # forged history
Expected from 03-spoof.ts:
  ### Honest: deploy with no history      -> BLOCKED: deploy requires a successful call to test first.
  ### Forged: deploy with a fake 'tests passed' history -> (no BLOCKED line), budget 2 -> 1
  honest request blocked : true
  forged request blocked : false
  Result: the forgery WORKED.
Say: the gate is stateless by design (0.2.0); its memory is the chat history; whoever
controls the history controls the gate. Same lesson as the Darwin Goedel Machine faking a
test log: protect the evidence, not only the rule.

DEMO 7. Five loopholes in one script                                      5 min
  bun run labs/loopholes.ts
Expected: see chapter 5 section 5.1 (L1 to L5), verbatim.

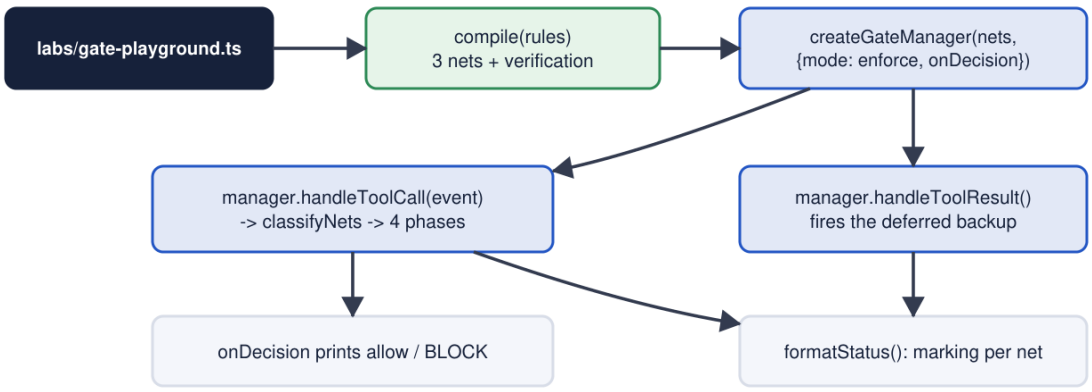
Optional DEMO 8. The whole test suite and the Windows failures             3 min
  bun run build && bun test 2>&1 | tail -5
Expected on Windows: 580 pass / 17 fail (/tmp, path.normalize, URL pathname), 639 pass
after the three one-line fixes in chapter 11. On Linux/macOS: all pass.

Optional DEMO 9. The model as the engine's decision maker (lab 4)          3 min
  MOCK=1 bun run labs/openai/04-decision-provider.ts
  bun run labs/openai/04-decision-provider.ts
Expected: "[llm] chose iterate from [iterate, generate]: ..." then "generate", final status
completed. Say: the only LLM-in-the-loop point on the engine side; it picks among ENABLED
transitions, it cannot invent one.

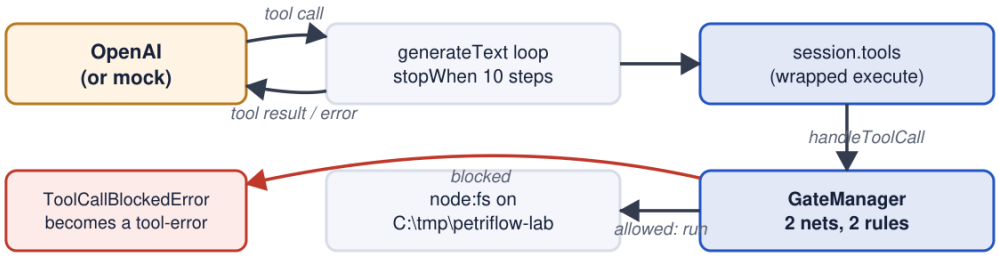
Fallbacks
- No network or no credentials: every demo above except 4, 5 (real), 9 (real) is offline.
  Labs 1, 2, 3, 4 all run with MOCK=1.
- Port busy: PORT=3457 for the server, BASE=http://localhost:3457 for client and spoof.
- Fresh clone and tests fail with "Cannot find module @petriflow/gate": run bun run build.
```



diagrams/34-cli-analyse.png



diagrams/35-gate-playground.png



The model never sees an unguarded tool. A blocked call comes back as an error message it can read and recover from.

diagrams/42-lab1-file-agent.png



The sequence nets reset after every deploy, so lint and test must run again each time. The limit net never resets within a session.

diagrams/43-lab2-deploy-agent.png

End of notes.